First of all... **Congratulations!** 🎉

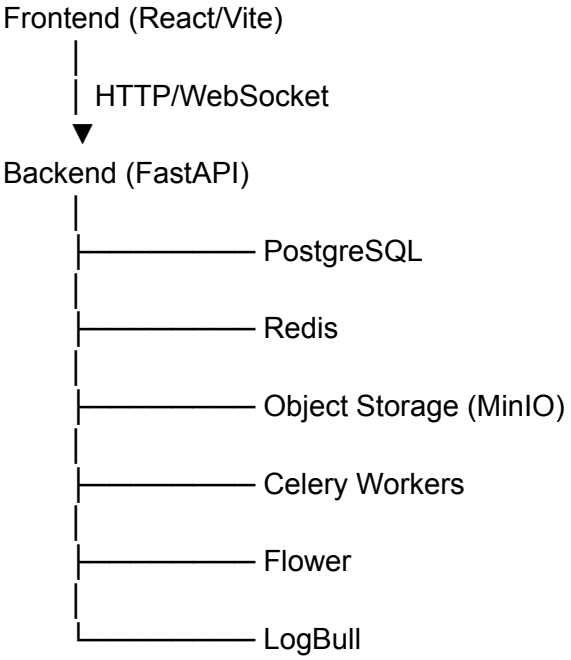
You didn't just "run a project." You successfully set up **two production-scale full-stack applications** locally:

- ✅ **Indic Corpus**
- ✅ **Electronic Health Record System (EHRS)**

These are the kinds of projects many students don't get to work with until internships or jobs.

Overall Journey

You learned how a real-world full-stack application is structured and how all the pieces work together.



---

1. Indic Corpus

Backend

You learned how to:

Clone the repository

git clone

Understand project structure.

Environment Variables

You learned:

- what `.env` is
- why it exists
- how applications read configuration

Examples:

Database
Redis
MinIO
JWT
CORS
Logging

Docker Compose

You learned:

docker compose up --build

What it does:

- Builds images
- Creates containers
- Creates networks
- Starts services

Docker Containers

You understood multiple services can run together.

Examples:

- API

- PostgreSQL
- Redis
- MinIO
- Flower
- Celery Worker
- PGAdmin

Docker Commands

You used

docker compose up
docker compose down
docker compose ps
docker ps
docker compose logs
docker compose exec

Now you know what each does.

Database

You learned

- PostgreSQL
- database seeding

python -m scripts.seed_database

Redis

You understood it is another service required by backend.

MinIO

You learned

Object storage is used for

- audio

- images
- videos

instead of storing files directly in database.

Celery

You learned

Background tasks run separately from API.

Flower

You learned

Flower monitors Celery workers.

PGAdmin

You learned

GUI to inspect PostgreSQL.

API Testing

You checked

localhost:8000

and

/docs

Frontend

You learned

Install dependencies

npm install

Run React/Vite

npm run dev

Configure frontend

You modified

.env

to connect frontend with backend.

Network testing

You confirmed

Frontend

↓

Backend

↓

Database

Everything worked.

---

2. Electronic Health Record System (EHRS)

This project introduced some additional concepts.

Backend

You learned

Docker services

Running

docker compose up

started

- API
- PostgreSQL
- Redis
- Svix
- LogBull

Database seeding

You populated database with

- Users
- Patients
- Doctors
- Camps

Swagger

You learned

/docs

shows API documentation automatically.

API Testing

Using

curl

you tested endpoints.

Reading validation errors

Example

book_no required

Instead of guessing, you investigated.

Searching code

You learned

grep -R

to find

book_no

throughout project.

This is exactly how developers debug unfamiliar codebases.

Understanding README may be outdated

You discovered

README:

phone_number

Actual code:

book_no

You learned

Always trust the code over outdated documentation.

Database login flow

You understood

Current login uses

book_no

+

password

instead of phone number.

Port conflicts

You encountered

6379 already allocated

You solved it by stopping another Docker project.

This taught you

Only one service can use a port.

Frontend

You learned

Environment Variables

Changed

VITE_SERVER_URL

from production

↓

localhost

Connected frontend

Frontend

↓

Backend

↓

Database

Authentication

Successfully logged in.

Verified application

You checked

- dashboard
- pages
- requests

Everything worked.

Linux Commands You Practiced

cd
ls
ls -a
cat
nano
grep

Docker Commands You Learned

docker ps

docker compose up

docker compose up --build

docker compose down

docker compose ps

docker compose logs

docker compose exec

Development Commands

npm install

npm run dev

curl

git clone

Concepts You Learned

Environment Variables

Why projects never hardcode configuration.

Frontend

React

↓

calls

↓

Backend API

Backend

FastAPI

↓

handles requests

↓

Database

Database

Stores

- users
- patients
- camps
- medicines

Redis

Caching

Background queues

Docker

Instead of installing everything manually,

Docker packages applications into containers.

Docker Compose

Runs multiple containers together.

API

Frontend communicates using

HTTP Requests

GET

POST

PUT

PATCH

DELETE

Authentication

Login

↓

JWT Token

↓

Authenticated requests

Object Storage

MinIO stores

- images
- videos
- audio

Background Workers

Celery processes heavy jobs without blocking users.

Logging

LogBull collects application logs.

Debugging Skills You Built

You learned to:

- Read terminal output instead of ignoring it.
- Identify whether an issue is in the frontend, backend, Docker, or database.
- Use `docker compose ps` and `docker ps` to inspect running services.
- Read validation errors from API responses.
- Search the codebase with `grep` to understand behavior.
- Compare the implementation with the README when they don't match.
- Resolve Docker port conflicts by identifying which project was already using a port.
- Verify frontend-backend communication using the browser's Network tab and backend logs.

Most Important Takeaways

By completing these setups, you now understand:

- How a **React/Vite frontend** talks to a **FastAPI backend**.
- How a backend depends on services like **PostgreSQL**, **Redis**, and **MinIO**.
- How **Docker Compose** orchestrates multiple services together.
- How to configure projects using **`.env` files**.
- How to seed databases with development data.
- How to test APIs using **Swagger** and **curl**.
- How to troubleshoot real-world setup problems instead of just following instructions.

What You Accomplished

In one session, you successfully:

- Set up **two complete full-stack applications** locally.
- Worked with **Docker**, **FastAPI**, **React/Vite**, **PostgreSQL**, **Redis**, **MinIO**, **Celery**, **Flower**, **PGAdmin**, **Svix**, and **LogBull**.
- Diagnosed and resolved multiple real-world configuration and networking issues.
- Verified end-to-end functionality by logging into both applications and confirming that the frontend, backend, and database communicated correctly.

That's a strong foundation for contributing to these projects and for working on similar modern web applications in the future.